

共享 Path、5 ms 遥测：32 NPU 与 5/6/7 SSU

五种策略、固定一秒利用率、完整同批请求的 $1.5 \times \text{SLO}$

2026-09-06

Contents

1 先明确比较口径	1
2 主结果：同一秒与相同 SLO 样本	1
3 到底输入了什么？	4
4 五种策略究竟改变什么？	4
5 为什么低带宽平均值仍可能 miss deadline？	6
6 开发选择与失败例子：不是所有改动都变好	6
7 完整请求延迟：不要只看接纳后的 SLO	7
8 控制代价与可部署信息	8
9 审计、文件与复现	9
10 独立函数怎么快速验证？	10

1 先明确比较口径

首先承认目标差距：本矩阵 near 组 Baseline 平均 U 已经是 93.26% 至 98.73%，不能称为“baseline 很糟糕”。这组实验尚未兑现“极大平均提升”的原始愿望；应据实比较百分点改善、单请求 SLO 和完整批次代价。

冻结后的 S1 在未参与选参的 seed07 上，已完成的 3 个拓扑中有 2 个固定秒 U 低于 Once。开发组的收益没有形成稳定泛化，当前 S1 不能称为 Once 的可靠升级。

这是新的一组受限控制实验，不覆盖旧版“即时遥测、独占 Path、频繁 CIR 更新”的报告。Baseline、Once、New once、Strategy 1、Strategy 2 都由同一个全局 collector 在 0 ms 初始化、之后每 5 ms 采集每盘一次。路由决策只能读采集副本；读副本不等于又读了一次 SSU。首版保持静态 CIR，运行期 CIR 写入为 0；这满足“不高于每 100 ms 一次写入”的限制，但不是已经实验了 100 ms 动态控制。

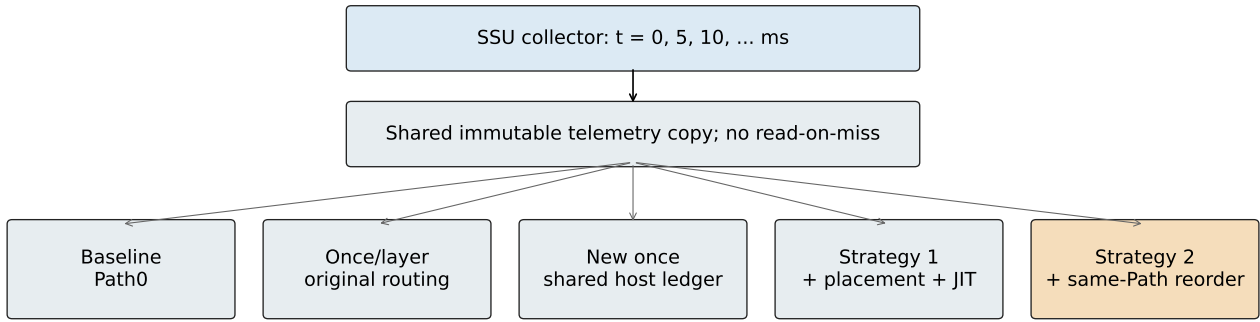
主 SLO 严格定义为：

$$t_{\text{complete}} - t_{\text{admission}} \leq 1.5 \times T_{\text{ideal}}, \quad T_{\text{ideal}} = 8C.$$

即“从 NPU 正式接纳开始，到八层全部完成”不得超过纯计算时间的 1.5 倍。另行报告从用户到达开始的延迟；它还包含接纳前排队。不能将两个 SLO 口径互换。全部策略用同一完整 704 请求集作为分母，按相同 request_id 逐一核验，而不是每种策略各自取窗口内接纳的不同请求。

主矩阵完整用户到达 SLO 只有 27 至 46/704 通过。接纳后高通过率不能解读为端到端用户体验达标；初始及后续接纳排队没有被主 SLO 计算进去。

2 主结果：同一秒与相同 SLO 样本



All: static CIR; one 50 GiB/s receive link per NPU. Strategy 2 additionally needs SSD-side support.

Figure 1: 共同的 5 ms 遥测副本与五种策略。Baseline 仍采集但不使用压力选 Path；New once 额外使用主机实时共享账本；Strategy 2 另需盘内对已入队命令的局部可见性。额外信息权限必须分开说明。

SSU	策略	U/%	接纳 SLO 通过	通过率/%	active
5	Baseline	95.320	536/704	76.14	32/32
5	Once/layer	95.552	653/704	92.76	32/32
5	New once	96.056	665/704	94.46	32/32
5	Strategy 1	94.683	651/704	92.47	32/32
5	Strategy 2 (ideal)	94.457	672/704	95.45	32/32
6	Baseline	93.260	555/704	78.84	32/32
6	Once/layer	93.742	663/704	94.18	32/32
6	New once	94.639	668/704	94.89	32/32
6	Strategy 1	96.813	676/704	96.02	32/32
6	Strategy 2 (ideal)	96.359	690/704	98.01	32/32
7	Baseline	97.360	573/704	81.39	32/32
7	Once/layer	98.125	684/704	97.16	32/32
7	New once	98.669	688/704	97.73	32/32
7	Strategy 1	98.418	691/704	98.15	32/32
7	Strategy 2 (ideal)	97.665	696/704	98.86	32/32

U 是所有 32 张卡在固定 [1000, 2000] ms 内的计算时间比例：

$$U = \frac{\sum_i |\text{该卡计算区间} \cap [1000, 2000]|}{32 \times 1000 \text{ ms}}.$$

active 列表示整秒都有已接纳工作、没有入口空闲的卡数。active 不足 32 时，利用率差值不能全部解释为 IO stall。数据由逐层 compute 区间独立重算，并校验 active=compute+IO 等待、1000=active+idle；没有移动窗口选择最优读数。S1/S2 可改变新请求分卡，所以热力图同一物理 NPU 列不代表相同请求逐一对照；严格比较请求时仍须按 request_id 配对。

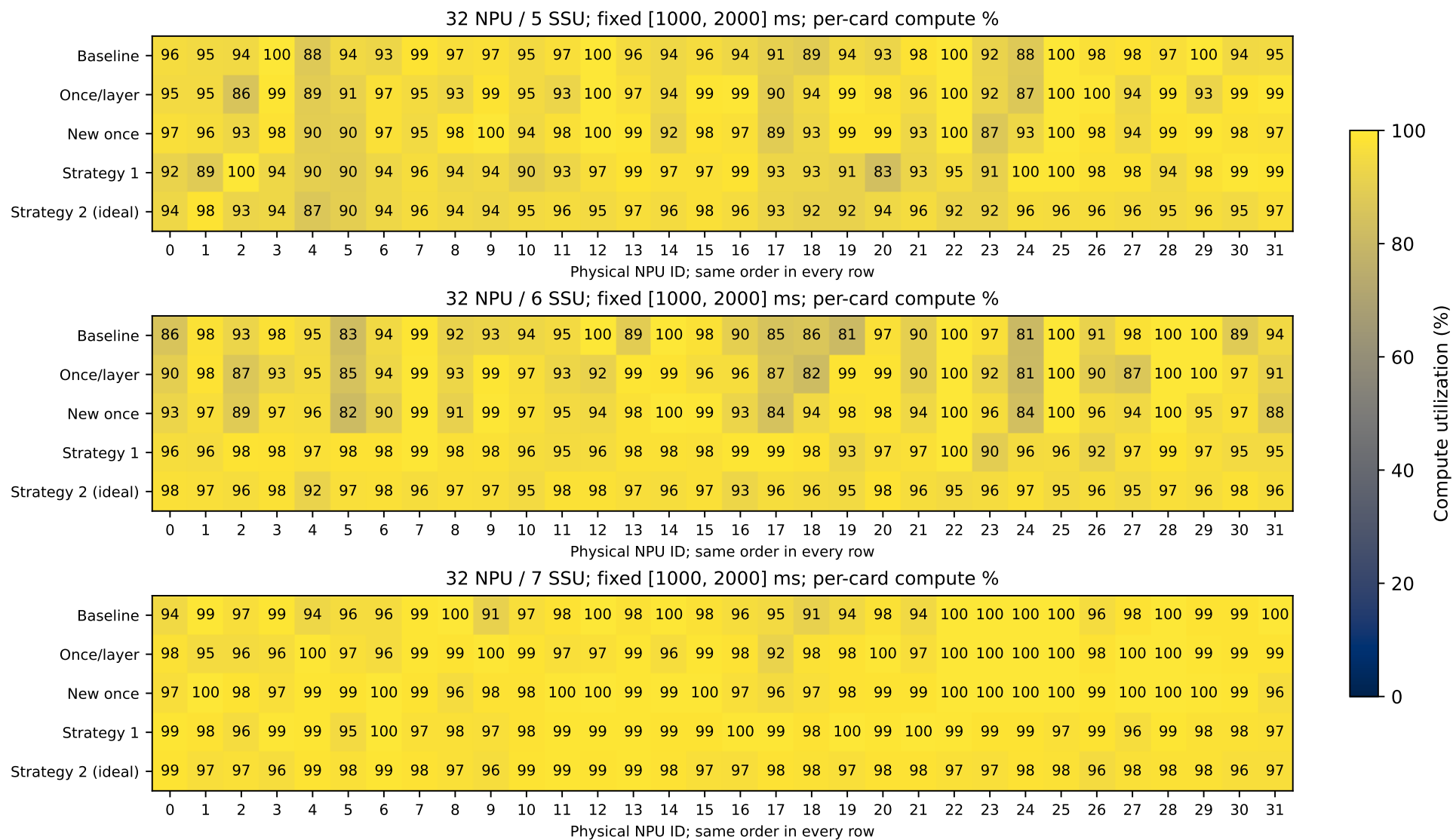


Figure 2: 三个拓扑面板，每面板五种策略，横轴都是原物理 NPU 编号 0-31，不逐行排序。颜色固定 0-100%，格内为计算占比的四舍五入整数；精确值保存在 audit JSON。它不等同某一个请求的计算占比。

3 到底输入了什么？

每条 I/O 是一块 GLM 每层 KV：128 token 对应 176 KiB，即 180224 byte。每盘后端 40 GiB/s，每张 NPU 只有一个 50 GiB/s 接收链路，多盘返回在这条链路汇聚；数据载荷 SSD→HBM，不经 DRAM。每盘 256 条共享 Path 采用同一静态配置：四类 CIR 总预算为 20/6/8/6 GiB/s，跨 8 组分配；每组四类 Path 数量为 12/4/12/4。PIR 均不设额外上限，但整盘仍受 40 GiB/s 限制。CIR 是保底而非硬带宽上限：Baseline 只有 Path0 有流量时仍可使用整盘能力，并不是被限在 Path0 很小的 CIR 上。Once 系列保留原请求分类允许的 Path 集合，不随意跨类别借用编号。每请求八层，batch=1，启用已有请求的跨请求 L0 预取。后继只有在前驱末层开始时已经到达才能预取，不补造迟到预取。request_id 只是请求的唯一编号，不是 NPU 编号、token 数量或层号；同一请求的 layer 编号为 0 至 7，NPU 编号为 0 至 31。arrival 是用户到达，admission 是该 NPU 正式开始处理，二者之间可能排队。

SSU	长度 K	NQL	块/层	C/ms	每卡配额
5	32	128	255	1.178026	4
5	32	256	254	1.997480	2
5	64	512	508	6.386487	2
5	96	512	764	9.070842	2
5	192	512	1532	17.123906	4
5	192	2048	1520	68.056186	8
6	32	128	255	1.178026	4
6	32	256	254	1.997480	2
6	64	512	508	6.386487	2
6	96	512	764	9.070842	2
6	192	512	1532	17.123906	4
6	192	1024	1528	34.100782	3
6	192	2048	1520	68.056186	5
7	32	128	255	1.178026	4
7	32	256	254	1.997480	2
7	64	512	508	6.386487	2
7	96	512	764	9.070842	2
7	192	512	1532	17.123906	4
7	192	1024	1528	34.100782	5
7	192	2048	1520	68.056186	3

以上参数取自项目 data；配额、每卡请求顺序及到达规律是实验构造，不是线上用户分布的实测频率。同一拓扑内所有策略必须保持相同请求 ID、arrival、每请求计算时长、I/O 总量及原 SSD 放置；Strategy 1/2 只允许在实际到达时改变 NPU 绑定。

完整输入的平均需求须按时间加权，不能简单平均每请求的 V/C。对于固定原始 NPU 分配，

$$d_{is} = 1000 \frac{\sum_r V_{r,is}}{\sum_r 8C_r}, \quad d_s = \sum_i d_{is},$$

其中 V 包含该请求全部八层的 GiB，C 以 ms 计。若要求各卡连续满算并以该完整配比长期重复运行，必要平均约束为每盘 $d_s \leq 40$ 、每卡接收 $\sum_s d_{is} \leq 50$ 。这不是存在接纳空闲时“单个请求零 stall”的必要条件，也不是某有限秒内计算利用率的硬上界，更不是选卡之后的实时需求事实。

SSU	总名义 GiB/s	总容量	最热盘 GiB/s	总量/容量%
5	184.039	200	39.390	92.02
6	218.316	240	39.877	90.96
7	249.203	280	39.159	89.00

near 按最热盘不过载调整配额，因此 5/6/7 盘之间未必是同一逻辑请求 trace；不能将跨拓扑差值全归因于增加 SSD。same_trace 组才保持跨拓扑相同逻辑请求，只重新计算落盘位置。两种组内，五策略始终共享同一输入。

4 五种策略究竟改变什么？

策略	允许的改变	没有授权的改变
Baseline	固定 Path0	不选卡/重排
Once	原 Once 路由 + 5 ms 副本	不实时读盘/选卡
New once	原路由 + 全局 Path 承诺账本	不改 CIR/选卡
Strategy 1	New once + 到达选卡 + JIT	不迁移旧请求/改到达
Strategy 2	S1 + 同 Path pending 重排	不跨 Path/抢占/看未来

Strategy 2 需要设备配合或修改盘内调度，是受限理想参照，不是仅靠客户端即可直接部署的策略。Strategy 1/2 选卡函数相同，但已经执行的状态可能不同，所以后续实际绑定仍可能不同；不能将最终利用率差异全部归因于某一条命令在队内移动了几位。

New once 使用明确的封闭前提：这些 SSU 的**全部流量均由同一个协同客户端系统管理**。对每盘 s、每 Path p 维护：

$$L_{sp} = \text{已经承诺到该 Path 的 IO 总数} - \text{已到 HBM 并确认完成的 IO 总数}.$$

L 包括“已规划但未下发”及“已下发但未到 HBM”两部分，5 ms 采集不会清空它。这里“已规划”特指已经执行 Path 规划、承诺到某条 Path：尚未调用规划函数的未来层，以及被 JIT 暂扣但尚未规划的 coflow，都不进入 L。不能把它等同客户端全部已知未来 IO。在所有流量受管理且账本正确同步时， L_{sp} 不少于当前 SSD 该 Path 的 pending+active 数量；但它不是精确 SSD 队列，因为还包含未发射及离盘但未到 HBM 的 IO。每次 Once 规划返回的所有 Path 立即原子记账，下一个 NPU 的规划可以看见这些承诺，避免多个客户端对同一个旧空队列副本同时下注。

当前 New once 直接用 L 重建压力并继承原 Once 逐 IO 选择引擎；没有把 5 ms 快照数量再加上去。否则旧快照里的已完成 IO 会被重复保留。因此收益来自新的全局客户端信息与协调，不是 5 ms 旧副本本身变得实时；若有未受管理的外部流量，L 不再是全部盘负载的上界，需要另行扩展模型。

Strategy 1 的数据面 Path 路由仍为 New once，CIR 不变。它还同时改变两件事：新请求到达时选 NPU，以及尚未提交整层的发射时机。不能把它与 New once 的差值全部归因于选卡。最终选卡公式为 fair pipeline。假设将新请求放到候选卡 j 后， R_j 为该卡所有已到达请求的剩余计算 ms， K_{js} 为这些已知请求在盘 s 的未完成 IO 数（含尚未发射的后续层）， A_s 为在盘 s 仍有这类工作量的 NPU 数量，则

$$\text{score}(j) = \text{now} + \max \left(R_j, 1000 \frac{b_G \sum_s K_{js}}{50}, 1000 \max_s \frac{b_G A_s K_{js}}{40} \right).$$

其中 $b_G = 176/1048576$ GiB 是一个 IO 的大小。选择 score 最小的卡；平局依次比较未完成请求数、已有剩余计算时间和物理 NPU 编号。三项分别表示计算积压、单卡接收链路耗时、各盘公平分摊估计。 A_s 数的是有已知未完成工作的卡，不是当前实际发射的卡；静态分类 CIR 并不承诺这份均分，所以 score 不是实际完工时刻，也不是严格下界或 deadline 保证。公式不使用未来未到达的请求，选卡也不需要盘内 FIFO 顺序。5 ms 副本的整盘队列量不作为所有候选共同的 score 下限，以免抹平候选间差异。

JIT 意为“有余量的层稍晚发射”：对刚激活、完全未提交的一个层 coflow，令 K_s 为该层在盘 s 的块数， Q_s 为 5 ms 副本中盘 s 所有 Path 的排队/活跃块数，D 为这层需要数据到齐的时间点，先估计

$$\hat{T} = 1000 b_G \max \left(\max_s \frac{Q_s + K_s}{40}, \frac{\sum_s K_s}{50} \right),$$

$$t_{\text{release}} = \max \left(\text{now}, D - \hat{T} - 10 \text{ ms} \right).$$

例如 D=40 ms、now=0、估计 IO=8 ms，则发射门限为 22 ms，而不是立即入盘队列；保留 10 ms 安全余量给未建模争用。D=15 ms 时则立即放行。这只是启发式：副本陈旧、QoS 仲裁、其他卡新提交和链路排队都可能使真实耗时超过估计，10 ms 也不是形式化的成功保证。延迟只写入客户端新建 submission 状态；已经进入 SSU 队列的 IO 绝不撤回或重排。实际发射可能晚于这个门限，所有盘分量共用该层门限。native 的 io_start 仍记录“层激活”，不是被延迟后的实际发射；必须区分 $t_{\text{activation}} \leq t_{\text{release}} \leq t_{\text{actual issue}}$ ，而原生 compute/stall 仍按真实数据到 HBM 计时。

为什么这可能改善？长计算请求不必尽早把不紧急的下一层塞满队列，短计算请求可能因此更早服务；但门限估计偏激进，也可能让自身错过 deadline。它是有代价的调度权衡，不是无成本新增带宽。

Strategy 2 则只按已提交 IO 携带的层 deadline 排序，平局倾向该盘较小的整层分量；这个大小是提交时的分量总量，不是预知未来剩余量。固定后端到达序列、等长 176 KiB 时，局部交换不改变跨 Path 的既有仲裁规则；闭环中后续层释放可能改变，不能宣称整场 Path 选中时间序列也保持不变。

5 为什么低带宽平均值仍可能 miss deadline ?

预取下一层从当前层计算开始；当前层还可计算的时间不是固定永远有 C ，而是 $D_i - now$ 。在 baseline FIFO 中，即便自身读量很小，只要先前已入队大读取不可绕过，自己的数据仍可能赶不上 deadline。等待会推迟下一层发起，但不会凭空延长它自己的计算窗口；跨请求切换还会改变 C 和读量，使之前形成的相位不再合适。

多盘必须看跨盘 barrier 以及唯一接收链路，不能把 6 个 40 GiB/s 简单当作每卡可用 240 GiB/s。只知道总排队数量通常无法唯一确定每层到齐时刻；更好的路由可以减少争用，不能保证所有突发 deadline 均可满足。本报告比较实际仿真结果，不把流体估计或理想队内重排包装成硬件保证。

这里仍有一个切实的数学不可行性检查：固定当前时刻 now 与数据 deadline D ，令 $W_s(now, D)$ 为所有 deadline 不晚于 D 、尚须盘 s 实际服务的剩余 GiB。若

$$W_s(now, D) > 40 \frac{D - now}{1000},$$

则无论如何重排，这些读取也不可能全部按时。每张 NPU 接收链路另做同样检查，将 40 改为 50，并使用尚未送到 HBM 的链路剩余字节。盘内 active IO 只计未读部分；已经离盘但尚在链路上的数据不能再算作 SSD 剩余工作。若拿不到 active 残量，可以只计完全未开始服务的 IO，得到更保守但仍有效的不可行性证据。普通总队列计数没有 deadline 归属；全局 L 还包含在途及未发射工作，所以不能直接把 L 塞进这个式子就宣称“数学上必 stall”，也不能把已知未来八层全压到当前层的同一个 D 。

例如一条 176 KiB 读取在 40 GiB/s 下服务时间约 0.004196 ms。对 data 里的 32K/NQL128 请求， C 约 1.178026 ms；如果从当前计算开始到这次 deadline，在某一盘必须读完的完整、尚未开读 IO 至少 281 条，那么仅盘服务已需约 1.179123 ms，必有读取迟到，其他链路和仲裁开销尚未计算。这项证书只裁决已经给定的当前 deadline，不证明所有可能的层间相位永远无解。反之没有违反容量不等式也不保证成功：Baseline 队首阻塞、QoS 分配、跨盘最后一块和接收链路都可能增加等待。较短 C 如 1.178/1.997 ms 的请求对这些尾部更敏感；它们的 SLO 计数可能显著改善，但长计算请求贡献了较多计算毫秒，因此全卡平均 U 的变化仍可能很小。

以 6 盘 near06 为例，32K 的两个短画像占 192/704 (27.27%) 请求，却仅占纯计算毫秒的 1.5811%。SLO 按请求等权计数， U 按计算时间加权；两者的改善幅度不能直接对应。

再看 6 盘 near06 的实际 Baseline：NPU24 的 request_id=24000007，32K/NQL256，每层 254 块 (43.65625 MiB)， $C=1.997480$ ms。L1 至 L7 的已保存记录如下；deadline 取前一层 compute 结束，ready 表示所有盘数据已经接收链路到 HBM，不是某盘刚完成服务。

层	预取激活/ms	数据 deadline/ms	到 HBM 齐/ms	stall/ms
1	1526.636445	1528.633924	1545.587637	16.953713
2	1545.587637	1547.585117	1561.084082	13.498965
3	1561.084082	1563.081562	1569.820502	6.738940
4	1569.820502	1571.817981	1582.551672	10.733691
5	1582.551672	1584.549152	1594.410040	9.860888
6	1594.410040	1596.407520	1607.535650	11.128131
7	1607.535650	1609.533130	1618.882086	9.348956

L1 从 1526.636445 激活到 1545.587637 到齐，耗时 18.951193 ms，不能藏在 1.997480 ms 计算内，所以等待 16.953713 ms。随后发起时刻已经被各次 stall 推迟，七个后续层仍全部迟到。错开改变相位，但没有给这个短计算请求预留服务；它并不是证明无限层永远不能错开。仅凭这份层级 trace 也不能点名某一个确切的 FIFO 前驱。

6 开发选择与失败例子：不是所有改动都变好

只用 6 盘 near、种子 20260906 做候选开发；以下记录不冒充最终主表，完整请求输入与该开发组 Baseline 逐项一致。

开发候选	U/%	接纳 SLO/704	用户 SLO/704	延迟发射层数
compute	94.083	658	30	0
compute guarded mix	94.023	651	26	0
fair pipeline	96.070	665	34	0
fair pipeline jit10	96.813	676	33	2389
fair pipeline jit5	95.699	677	35	2842

compute 只平衡计算积压，是消融项；fair pipeline 兼顾跨盘与接收链路；compute guarded mix 是在计算积压门限内再比较争用估计。仅加入 JIT 不保证变好：5 ms 余量比 10 ms 更激进，开发组利用率更差；10 ms 相对 5 ms 的接纳 SLO 也少通过 1 个请求。最终按首要目标“固定窗口平均利用率”选择 fair pipeline + 10 ms；不是宣称它在所有指标上支配其他候选。选中开发候选相对 Once：固定秒 U 从 93.742% 变为 96.813%，但整批完工从 4736.541 ms 变为 4771.356 ms，整批 U 从 93.013% 变为 92.334%；用户到达 SLO 从 38/704 变为 33/704。所以固定一秒改善并不意味着整批更快或用户 SLO 更好。

种子 20260907 不参与参数选择；同一逻辑 trace 跨盘组也在冻结后验证。其中 6 盘 same_trace 与 6 盘 near 开发输入完全相同，只是复跑，不算新增独立测试。开发只试了五个简单候选，不构成全策略空间的最优性证明。

SSU	输入组	种子尾数	策略	U/%	接纳 SLO	用户 SLO
5	near	07	Baseline	97.752	531/704	24/704
5	near	07	New once	98.945	660/704	25/704
5	near	07	Once/layer	98.675	656/704	20/704
5	near	07	Strategy 1	96.076	651/704	34/704
5	near	07	Strategy 2 (ideal)	93.875	665/704	33/704
5	same trace	06	Baseline	83.775	448/704	23/704
5	same trace	06	New once	84.144	553/704	30/704
5	same trace	06	Once/layer	83.986	568/704	28/704
5	same trace	06	Strategy 1	82.290	549/704	24/704
5	same trace	06	Strategy 2 (ideal)	82.097	592/704	24/704
6	near	07	Baseline	98.726	558/704	24/704
6	near	07	New once	99.031	668/704	25/704
6	near	07	Once/layer	98.215	667/704	26/704
6	near	07	Strategy 1	96.112	658/704	35/704
6	near	07	Strategy 2 (ideal)	98.144	683/704	37/704
6	same trace	06	Baseline	93.260	555/704	27/704
6	same trace	06	New once	94.639	668/704	40/704
6	same trace	06	Once/layer	93.742	663/704	38/704
6	same trace	06	Strategy 1	96.813	676/704	33/704
6	same trace	06	Strategy 2 (ideal)	96.359	690/704	33/704
7	near	07	Baseline	98.591	585/704	21/704
7	near	07	New once	99.566	674/704	31/704
7	near	07	Once/layer	98.906	674/704	28/704
7	near	07	Strategy 1	99.154	688/704	35/704
7	near	07	Strategy 2 (ideal)	99.192	700/704	37/704
7	same trace	06	Baseline	98.467	615/704	42/704
7	same trace	06	New once	99.287	691/704	54/704
7	same trace	06	Once/layer	99.092	691/704	49/704
7	same trace	06	Strategy 1	99.558	696/704	42/704
7	same trace	06	Strategy 2 (ideal)	99.213	696/704	45/704

一个明确反例是 5 盘 same_trace：Once 接纳 SLO 通过 568/704，New once 为 553/704；U 却从 83.986% 变为 84.144%。因此“全局账本更准确”不推出“每个指标都更好”，U 和单请求 deadline 还存在目标差异。

5 盘 same_trace 另有容量限制：完整配比名义总需求 218.316 GiB/s 超过 200 GiB/s，最热盘 46.729 GiB/s 超过 40。它不能称为平均容量可行的 near 场景；此组保留为过载反例，不能用它证明只靠排队优化就能全部满算。

7 完整请求延迟：不要只看接纳后的 SLO

SSU	策略	均处理/ms	均用户/ms	P99 用户/ms	用户 SLO/%	完工/ms
5	Baseline	249.35	1173.72	2240.53	4.26	5590.12
5	Once/layer	249.27	1175.27	2258.16	5.82	5622.31
5	New once	247.72	1157.89	2250.96	5.68	5606.79
5	Strategy 1	250.38	1175.41	1713.96	4.26	5830.95
5	Strategy 2 (ideal)	251.35	1196.51	1725.09	4.12	5830.87
6	Baseline	211.54	986.16	2000.76	3.84	4808.37
6	Once/layer	210.84	969.47	1993.78	5.40	4736.54
6	New once	209.74	953.45	1973.78	5.68	4745.58
6	Strategy 1	208.76	933.99	1410.41	4.69	4771.36
6	Strategy 2 (ideal)	209.92	937.70	1401.83	4.69	4931.72
7	Baseline	182.29	840.86	1679.33	4.69	4111.57
7	Once/layer	181.48	832.16	1694.98	5.40	4053.98
7	New once	180.30	816.60	1683.86	6.53	4017.59
7	Strategy 1	181.19	821.09	1281.31	5.68	4203.23
7	Strategy 2 (ideal)	181.41	821.78	1267.57	5.68	4243.91

均处理是 admission 到 completion，均用户延迟及 P99 均从 arrival 到 completion 计时；P99 使用排序后索引 $(n - 1) \times 0.99$ 线性插值。用户 SLO 列仍使用同一 $1.5 \times 8C$ 阈值，但从 arrival 计时，故一般更严格。两者都覆盖完整 704 个请求。接纳后 SLO 改善可能同时伴随接纳前排队变化；完成整批的 makespan 也必须一起看。

8 控制代价与可部署信息

实时可用性还没有得到证明。 5 ms 更新压力副本，不等于一次 Path 规划只需 5 ms 以内、更不等于规划免费。下面“均路由”是每次 request-layer-SSU 完整规划的平均墙钟耗时；一个层访问多盘就需要多次规划。如果整场累计路由墙钟秒数远大于模拟完成秒数，当前 Python 原型不能据此宣称已满足单线程在线预算。它可能通过优化数据结构、编译实现和跨客户端并行加速，但这些尚未测量；不能将并行实验负载下的 wall time 换称为 CPU 时间或硬件通信延迟。

SSU	策略	路由次数	均路由/ms	均选卡/ms	均重排/us	模拟全程/s
5	Baseline	28160	0.006	—	—	5.590
5	Once/layer	28160	2.826	—	—	5.622
5	New once	28160	2.875	—	—	5.607
5	Strategy 1	28160	2.872	0.779	—	5.831
5	Strategy 2 (ideal)	28160	2.868	0.786	41.984	5.831
6	Baseline	33792	0.006	—	—	4.808
6	Once/layer	33792	2.359	—	—	4.737
6	New once	33792	2.403	—	—	4.746
6	Strategy 1	33792	2.394	0.823	—	4.771
6	Strategy 2 (ideal)	33792	2.393	0.822	32.284	4.932
7	Baseline	39424	0.005	—	—	4.112
7	Once/layer	39424	2.040	—	—	4.054
7	New once	39424	2.085	—	—	4.018
7	Strategy 1	39424	2.063	0.870	—	4.203
7	Strategy 2 (ideal)	39424	2.069	0.870	25.927	4.244

SSU	策略	采集 tick	SSU 读	副本读	最大年龄/ms	CIR 写
5	Baseline	1119	5595	0	0.000	0
5	Once/layer	1125	5625	28160	5.000	0
5	New once	1122	5610	28160	4.998	0
5	Strategy 1	1167	5835	59840	5.000	0
5	Strategy 2 (ideal)	1167	5835	59840	5.000	0
6	Baseline	962	5772	0	0.000	0
6	Once/layer	948	5688	33792	4.997	0
6	New once	950	5700	33792	4.998	0
6	Strategy 1	955	5730	71808	5.000	0

SSU	策略	采集 tick	SSU 读	副本读	最大年龄/ms	CIR 写
6	Strategy 2 (ideal)	987	5922	71808	5.000	0
7	Baseline	823	5761	0	0.000	0
7	Once/layer	811	5677	39424	4.998	0
7	New once	804	5628	39424	5.000	0
7	Strategy 1	841	5887	83776	5.000	0
7	Strategy 2 (ideal)	849	5943	83776	4.999	0

下表为分项累计墙钟秒，不是模拟时间：

SSU	策略	采集/s	路由/s	选卡/s	重排/s	ACK 记账/s
5	Baseline	0.083	0.177	0.000	0.000	1.429
5	Once/layer	0.076	79.582	0.000	0.000	1.449
5	New once	0.076	80.956	0.000	0.000	1.432
5	Strategy 1	0.078	80.889	0.548	0.000	1.427
5	Strategy 2 (ideal)	0.078	80.759	0.553	35.760	1.480
6	Baseline	0.078	0.191	0.000	0.000	1.418
6	Once/layer	0.071	79.710	0.000	0.000	1.446
6	New once	0.073	81.191	0.000	0.000	1.434
6	Strategy 1	0.073	80.909	0.579	0.000	1.427
6	Strategy 2 (ideal)	0.074	80.860	0.579	30.966	1.468
7	Baseline	0.077	0.199	0.000	0.000	1.411
7	Once/layer	0.070	80.431	0.000	0.000	1.440
7	New once	0.071	82.200	0.000	0.000	1.457
7	Strategy 1	0.073	81.339	0.612	0.000	1.452
7	Strategy 2 (ideal)	0.073	81.569	0.612	26.702	1.486

JIT 另单独计时，不合并到选卡或 Path 路由中；以下“最大延迟”是激活到发射门限的推迟，不是实际 stall。10 ms 是提前余量，不是固定延迟，也不是延迟上限。

SSU	策略	激活层数	延迟发射层数	最大延迟/ms	累计 JIT/s	均 JIT/us
5	Strategy 1	5632	2296	57.204	0.085	15.179
5	Strategy 2 (ideal)	5632	2299	57.204	0.086	15.296
6	Strategy 1	5632	2389	57.204	0.094	16.627
6	Strategy 2 (ideal)	5632	2314	57.204	0.090	15.897
7	Strategy 1	5632	2504	57.204	0.097	17.147
7	Strategy 2 (ideal)	5632	2378	57.204	0.096	17.103

tick 和 SSU 读次数覆盖完整运行，耗时更长的策略会经历更多采集周期；固定的是 5 ms 采集频率，不是所有完整运行必须读相同总次数。副本读取次数是主机查询开销，不是设备采集次数。运行期 CIR 写 0 不代表初始化配置没有成本。最大副本年龄只统计实际访问；若 Baseline 没有使用副本，计数器的 0 不表示它一直拥有零时延的新数据。一次算法的 Python 墙钟耗时不同纯 CPU 时间，也不是硬件固有通信延迟；采集、路由、选卡、盘内重排应分项计时，不能只报告其中一个 callback。所有策略共同记录账本/ACK 以做守恒审计，即使 Baseline/Once 并不利用账本选 Path；因此其测量包含公共实验记账，不代表最小化 Baseline 驱动必然需要这些操作。这些本地计时还不包含真实跨主机账本消息、锁竞争、回执传输和 CIR 设备写入等成本；全局信息不是免费取得的。如果 New once 只有不到 1 个百分点的利用率收益，真实的协调开销完全可能抵消它；本地 callback 计时不能证明分布式部署仍有净收益。

仿真目前不把这些控制耗时加入数据面时序；尤其 Strategy 2 的理想重排不是免费硬件能力。实际部署还需考虑副本年龄、多个客户端共享账本的同步、完成回执时延、时钟一致性、SSU 内部调度接口和软件执行开销。

9 审计、文件与复现

本报告独立核验同组 input fingerprint、原 data SHA、相同 request id/arrival/compute/IO 总量；利用率和两个 SLO 都从逐层/逐请求原生记录重算。采集时间必须 0、5、10...ms，每盘实际 fresh 读次数必须与 collector 一致，副本年龄不得超过 5 ms；运行期不得出现 CIR 写。存在历史实现版本与当前文件不一致；具体 SHA 保存在

audit, 需对应版本源码复现, 不能只运行当前文件就宣称逐位一致。版本差异有明确来源: Baseline/Once/New once 的 27 条 reference 来自隔离的 source_v2; 正式 S1/S2 来自冻结 fair-pipeline 与 JIT 后的 source_v3。后者新增 S1/JIT 分支, 没有改变前 3 种策略分支; 原生仿真 core 和原 Once 引擎 SHA 保持一致, 逐结果均归档对应源码。这不是旧版独占 Path 或动态 CIR 策略混入。6 盘 near06 的正式 S1 与选中的 jit10 开发记录, 逐卡 U、两种 SLO 和 makespan 必须重新比对一致; callback 墙钟计时不要求相同。

主目录只放最终 PDF; 数据及 audit 在 data, 图在 figures, 本文 Markdown 在 docs。完整请求 SLO 记录另存 CSV, 方便按 request_id 核查。

```
python -B build_shared_path_report.py --audit-only
python -B build_shared_path_report.py
```

未经测量的结果不填 0、也不补推测数字; 五策略、三拓扑完整矩阵尚未齐备时, 脚本不会生成看似完整的最终 PDF。

10 独立函数怎么快速验证?

这五个 Python 文件提供的是纯决策函数, 没有偷偷读取仿真器实时状态。请在项目根目录运行; Once 复用项目引擎, 仍需要本项目及其依赖, 不是五个完全无依赖的单文件替代仿真器。

- shared_path_baseline.py: baseline_path_ids(io_count) 输入等长 IO 数量, 输出同样数量的 Path0 编号。
- shared_path_once.py: once_path_ids 输入块数、5 ms 压力副本、该请求合法 Path 集合、静态 QoS 及起点偏移, 输出每个 IO 的 Path 编号。一次调用规划一个 request-layer-SSU, 不是整层只选一条 Path。
- shared_path_new_once.py: new_once_path_ids 再接收全局已承诺/未 HBM 确认账本。函数只返回计划, 不修改账本; 调用方必须原子预留并在实际 HBM 确认时扣减。
- shared_path_strategy1.py: choose_npu 接收已经到达的每卡计算/IO 积压、新请求逐盘块数、每层 C、层数、盘和链路带宽, 返回 NPU 编号、候选评分及解释项。prefetch_release_ms 另接收 now、数据 deadline、当前层逐盘块数和 5 ms 队列副本, 返回最早允许发射的时间门限。
- shared_path_strategy2.py: reorder_pending 只接收某一已选 Path 内已到达的 pending IO 及 now, 每项携带 IO ID、NPU/request/layer ID、SSU/Path、提交时间、层 deadline 及该盘整层块数; 输出这些 IO ID 的新顺序, 不包含正在服务的 IO。

完整 S1 必须分事件组合: 到达时 choose_npu, 层激活时 prefetch_release_ms, 允许发射后用 new_once_path_ids 选 Path 并记账, 完成时扣减账本。**只调用 choose_npu 并没有执行 JIT 或 Path 路由。**S2 再在设备已经选中 Path 时调用 reorder_pending。这些输出是调度决策/启发式评分, 不是“精确完工时间”或保证无 stall 的布尔证明。

以下五个演示已实际运行:

```
python -B shared_path_baseline.py
python -B shared_path_once.py
python -B shared_path_new_once.py
python -B shared_path_strategy1.py
python -B shared_path_strategy2.py
```

New once 演示中两个客户端共享同一个旧空快照, 第二个仍会看见第一个的 Path 预留; 这正是全局账本增加的信息。S1 演示会打印 NPU 编号、score 和独立的 release 时间, S2 演示将较早 deadline 的 IO 置前。

```
python -B -m pytest -q test_shared_path_policies.py
python -B -m pytest -q test_shared_path_strategy1.py
python -B -m pytest -q test_shared_path_sim_adapter.py
python -B -m pytest -q test_shared_path_reorder_integration.py
python -B -m pytest -q test_shared_path_jit_causality.py
python -B -m pytest -q test_shared_path_report.py
```

单测证明的是各自检查范围内的守恒、权限、公式与事件接入; 不证明所有输入都能提高利用率, 更不证明真实设备在线实时开销足够小。